

## LECTURE 13

# ER Modeling: Issues and Problems

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

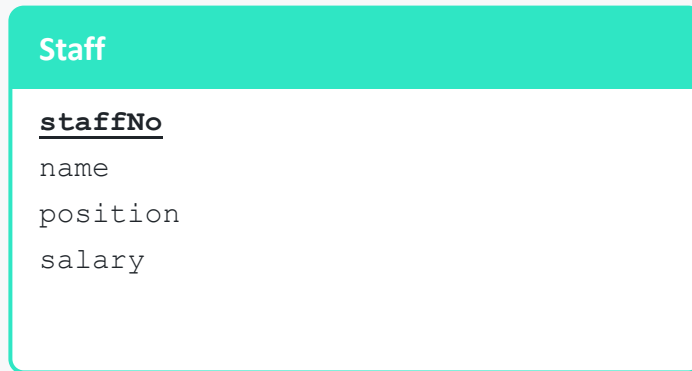
# In This Lecture

- Choosing entity types vs. attributes — the single most common beginner confusion
- Choosing relationship types, and spotting one that's hiding a missing entity
- Choosing attributes: simple, composite, single-valued, multi-valued, derived
- Identifying keys when more than one candidate key exists
- Naming conventions that keep a growing schema readable
- Complex relationships: ternary relationships, relationships with their own attributes
- Structural constraints revisited — the full (min, max) multiplicity notation
- Avoiding redundancy · a first look at generalization/specialization
- Two classic, named ER modeling errors: fan traps and chasm traps

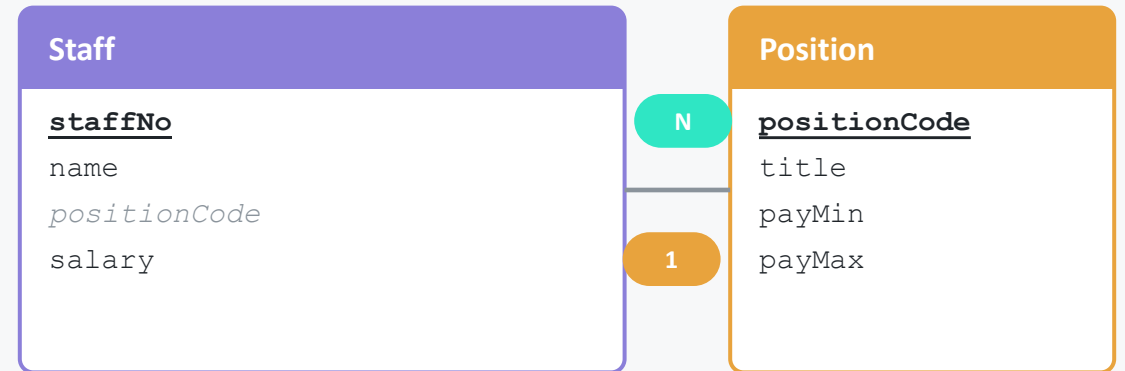
# Choosing Entity Types vs. Attributes

Should “position” be an attribute of Staff, or should Position be its own entity type? Both diagrams below are syntactically valid — only one fits a given set of requirements.

Option A — position as a plain attribute



Option B — Position promoted to an entity type



## The general rule of thumb (four questions)

1. Does it have its own attributes the business cares about? 2. Is it independently meaningful — does it relate to OTHER entity types? 3. Can more than one instance apply to the same real-world thing (own identity/lifecycle)? 4. Would leaving it as an attribute duplicate the same fact across many rows?

If yes to any: model it as an entity type. If it's a single atomic fact describing only one entity with no independent existence, keep it as an attribute. There is no permanently correct answer — re-check whenever requirements grow (e.g. city on Branch, if population/tax-rate tracking is later needed).

# Choosing Relationship Types

A relationship type is usually named from a verb phrase connecting two entity types (Staff Manages PropertyForRent). Three recurring issues:

## Redundant naming direction

Name the relationship once, in one clear direction. Adding “IsManagedBy” as a SECOND separate relationship type just models the same fact twice.

## A relationship hiding an entity

If a “relationship” needs its own attributes describing neither participant individually, it usually deserves its own identity — see Complex Relationships below.

## The wrong degree

Binary (two entity types, most common), unary/recursive (Staff Supervises Staff), or ternary+ (three or more at once). Force-fitting a 3-way fact into two binary relationships can silently lose information.

### Preview — this is exactly the fan trap mechanism

Collapsing a genuine three-way fact into pairwise binary relationships can lose the combination information entirely. This lecture's Fan Trap section (later) shows precisely how and why.

# Choosing Attributes

Once an entity type is settled, each attribute gets its own classification — this changes how it maps to a relational schema (Lecture 16).

| Kind          | Meaning                                       | Example (Client)       |
|---------------|-----------------------------------------------|------------------------|
| Simple        | Cannot be broken down further                 | telNo                  |
| Composite     | Made of sub-parts sometimes needed separately | name → first/last      |
| Single-valued | Exactly one value per occurrence              | dateRegistered         |
| Multi-valued  | Can hold several values at once               | preferredContactTimes  |
| Derived       | Computed, not stored itself                   | age (from dateOfBirth) |

Client — mixed attribute kinds

clientNo  
firstName  
lastName  
telNo  
preferredContactTimes {}  
dateOfBirth

**Derived attributes usually aren't stored**

age goes stale the moment time passes without an update. Most ER diagrams omit derived attributes entirely (computing age from dateOfBirth in a query) or mark them distinctly, precisely so nobody accidentally maps them to a stored column during Lecture 16's mapping step.

# Identifying Keys

An entity type can have more than one attribute (or combination) capable of uniquely identifying it — each is a candidate key. Choosing the primary key follows practical guidelines:

- Prefer the candidate key that never changes over the entity's lifetime
- cnic is stable; a badge number reissued on transfer is not
- Prefer the shorter, simpler candidate key when several qualify
- A single-attribute key over a composite one, all else equal
- Prefer a key with no NULL-able components, ever
- Entity integrity (Lecture 6) forbids NULL primary key components
- The remaining, unchosen candidate keys don't disappear
- They stay in the schema as alternate keys, usually enforced with UNIQUE

## Staff — two candidate keys

### staffNo

```
cnic  (alt. key, UNIQUE)
name
salary
```

### **staffNo chosen as PK**

Both staffNo and cnic are independently unique. staffNo is chosen as the primary key (short, stable, no NULLs); cnic remains an alternate key enforced by UNIQUE.

# Naming Conventions

A schema with a dozen entity types and fifty attributes is only as usable as its names. Two failure modes recur constantly:

## Synonyms

The same real-world concept named differently in different places — `clientNo` vs `custId`, both meaning “client identifier.” Makes joins and onboarding harder.

## Homonyms

The SAME name reused for two different concepts — “type” meaning House/Flat on `PropertyForRent`, but Full-time/Part-time on `Staff`. No clue which, read in isolation.

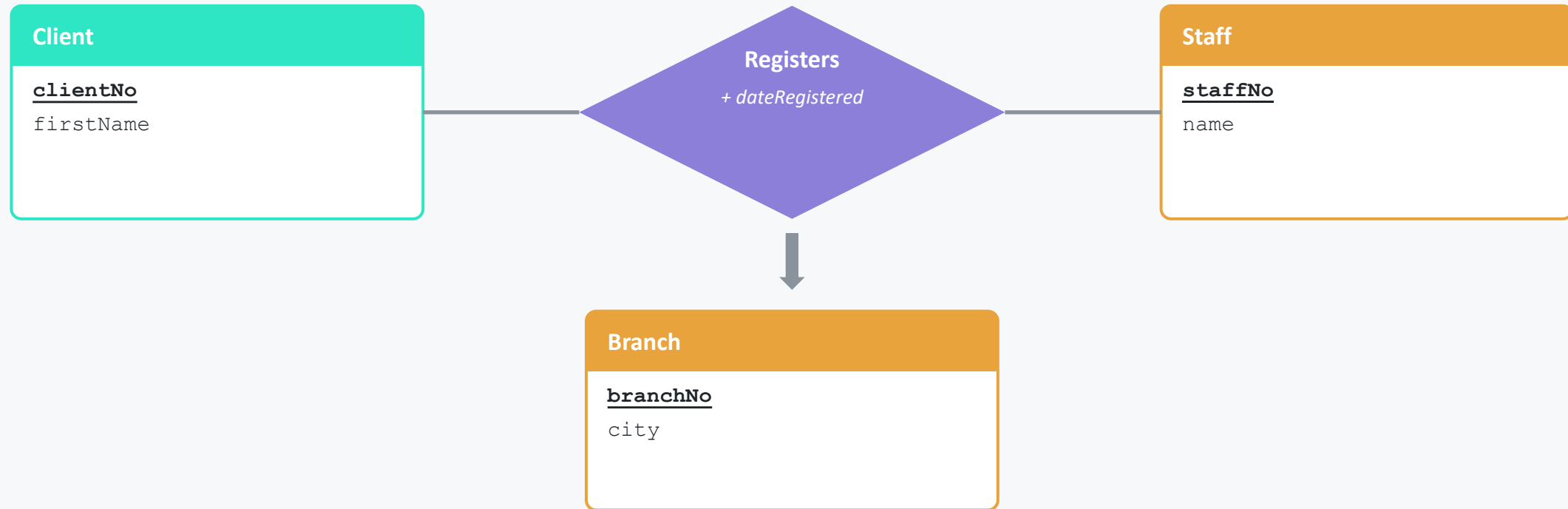
### **A practical naming standard, applied consistently across this course's schema**

Entity type names: singular, PascalCase (`Staff`, not `staffs`).

Attribute names: camelCase, prefixed with a short recognizable stem where it disambiguates (`staffNo`, `branchNo`, `propertyNo`) — every foreign key carries the SAME name as the primary key it references, exactly as `PropertyForRent.staffNo` does — which is precisely what makes a foreign key immediately recognizable on sight.

# Complex Relationships: Ternary

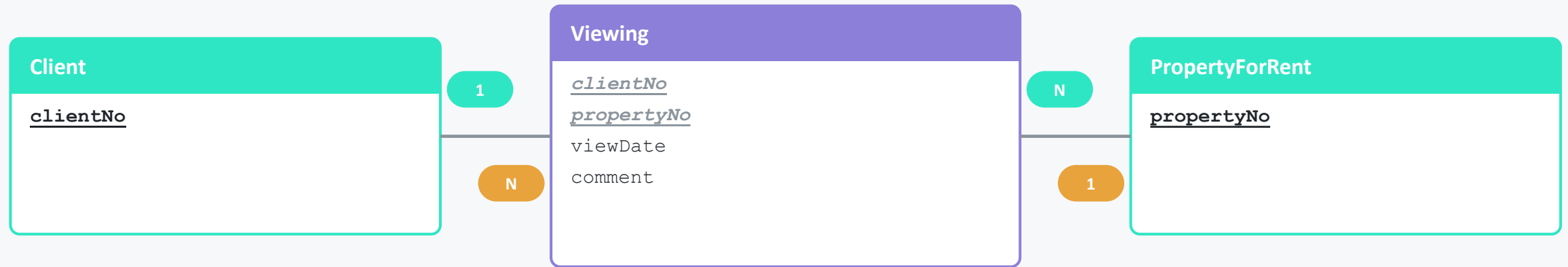
A ternary relationship connects three entity types at once — NOT generally the same as three separate binary relationships between the same pairs. Registers: a Client registers interest with a specific Staff member, at a specific Branch, on a given date.



Two binary relationships (Client Registers-With Staff, Client Registers-At Branch) would no longer guarantee those two facts describe the SAME registration event if a client interacts with the agency more than once. The ternary relationship keeps the combination intact.

# Relationships with Attributes

Viewing is the clearest example already in this course's schema: a Client views a PropertyForRent, and the relationship itself carries viewDate and comment — facts about that specific viewing event, not about either participant alone.



## Mapped to its own relation (Lecture 16)

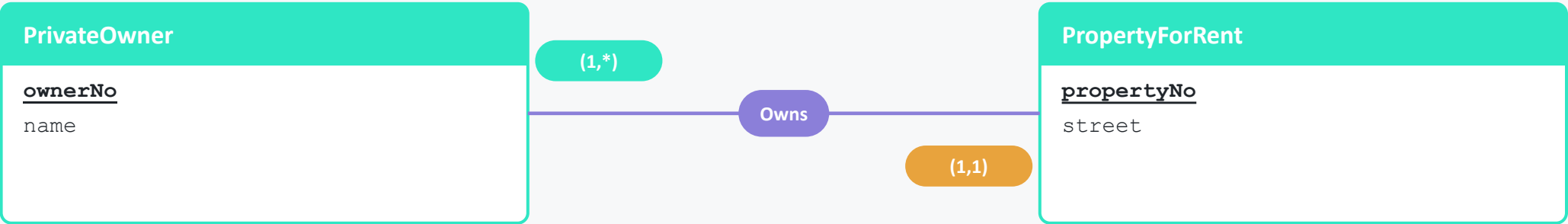
When a relationship needs its own attributes like this, it is mapped to its own relation:

Viewing(clientNo, propertyNo, viewDate, comment) — the underlined pair is the composite primary key, each component also a foreign key back into Client and PropertyForRent.

# Structural Constraints Revisited

Cardinality (1, N) and participation (mandatory/optional), combined, are written as a multiplicity pair (min, max) on each end of a relationship.

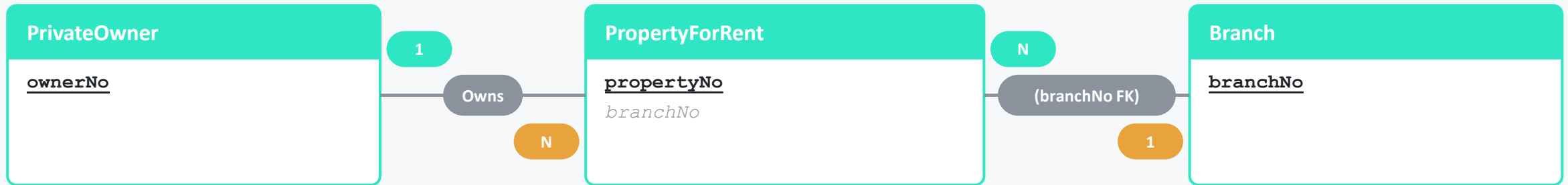
| Multiplicity | Meaning                               | DreamHome example                                   |
|--------------|---------------------------------------|-----------------------------------------------------|
| (0, 1)       | Optional, at most one                 | A PropertyForRent may have staffNo = NULL           |
| (1, 1)       | Mandatory, exactly one                | Every PropertyForRent belongs to exactly one Branch |
| (0, *)       | Optional, many allowed                | A Staff member may currently manage zero properties |
| (1, *)       | Mandatory, at least one, many allowed | A PrivateOwner must own at least one property       |



Getting these two numbers wrong in either direction is a real, common design bug — writing (0,1) on the owner side would incorrectly claim a property can legally have no owner at all.

# Avoiding Redundancy

A relationship is redundant if it conveys a fact already fully derivable by following a different path in the model. Test: delete this relationship — can I still answer the same question by tracing through others?



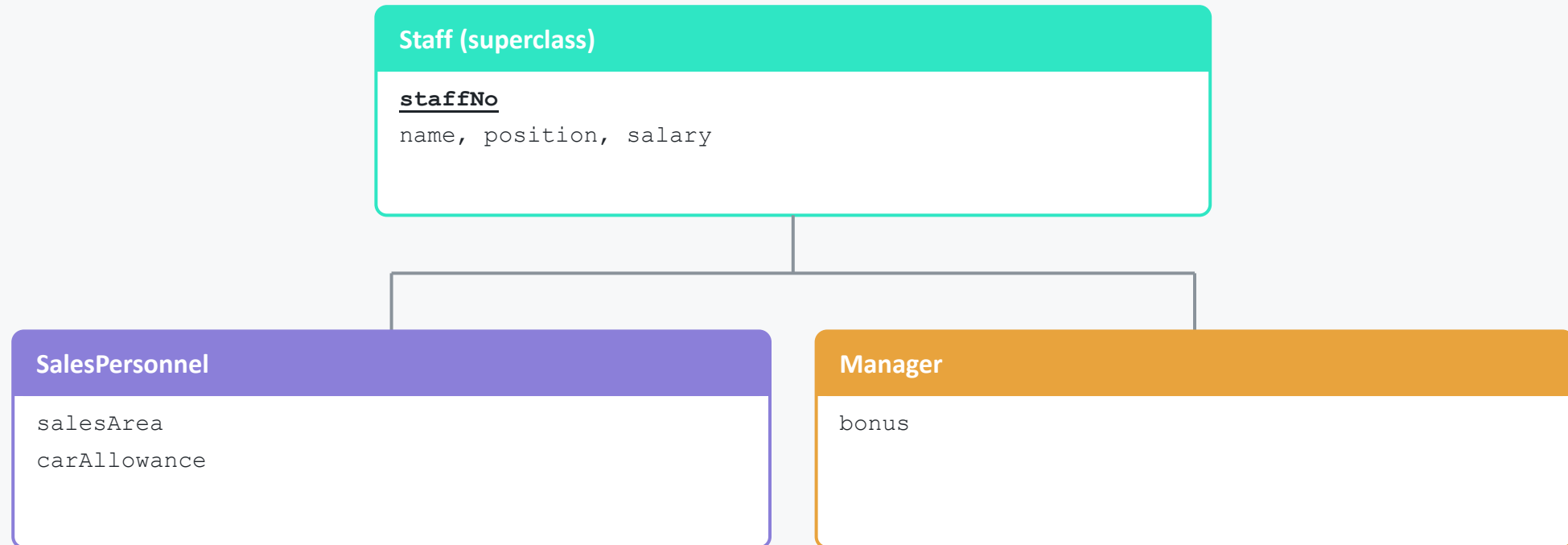
*Registers (redundant) — PrivateOwner → Branch, directly*

## Fix: delete the duplicate, don't try to keep two copies in sync

If **PropertyForRent** already carries **branchNo** directly, “which branch handles property P's paperwork” is already answerable — a third relationship claiming owners register with branches adds nothing but a second, possibly inconsistent, place for the same fact to live. Note: relationships that merely share an entity type (**Manages** and **Owns** both touch **PropertyForRent**) are NOT automatically redundant — only when the SAME fact is representable two different ways.

# A First Taste of Generalization / Specialization

Staff at a growing agency needs to distinguish SalesPersonnel (salesArea, carAllowance) from Manager (bonus) — both still Staff, sharing staffNo/name/position/salary, but each needs attributes the OTHER doesn't.



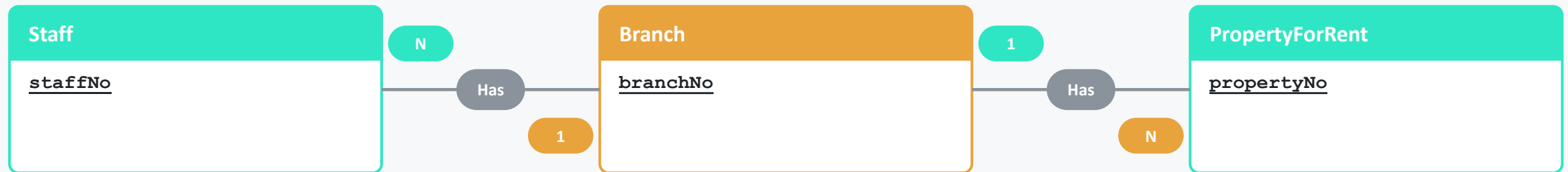
## Plain ER has no clean way to express this

Forcing it into plain ER means cramming all attributes onto one Staff entity (NULL for most rows — a red flag) or duplicating shared attributes across two unrelated entities. This gap is exactly what Lecture 14's Enhanced ER Model exists to close.

# Fan Trap

## Two 1:N relationships fanning out from the same entity

A fan trap occurs when two or more 1:N relationships radiate from a SINGLE entity type, and a question needs to relate occurrences on the “many” side of one to the “many” side of the other.



“Which properties does staff member SG14 manage?” cannot be answered — knowing Branch B003 has staff {SG14, SG37} and properties {PA14, PG4, PG36} doesn't say which staff manages which property.

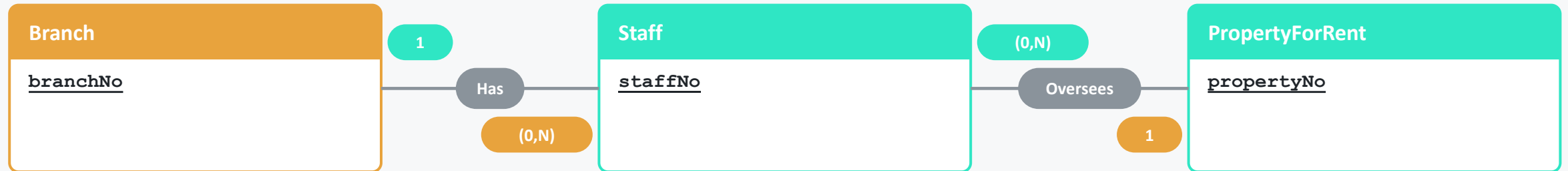
## Fix: add a direct relationship

Add Staff Manages PropertyForRent directly, so PropertyForRent.staffNo points straight at the responsible staff member instead of forcing a guess through Branch. This is exactly what Lecture 6 already does in this course's schema.

# Chasm Trap

## A path exists on paper, but breaks for some occurrences

A chasm trap occurs when a model implies a relationship along a path that includes an OPTIONAL (partial-participation) relationship — so the path exists for some occurrences but is genuinely missing for others.



“Which properties belong to Branch B009?” fails via Branch → Staff → PropertyForRent: if B009's staff all currently oversee zero properties, or B009 has zero staff, the path disappears entirely — even though B009 plausibly still has properties waiting to be assigned.

## Both traps share one root cause and one fix

Fan trap: two 1:N relationships fanning OUT of a shared entity. Chasm trap: a 1:N/optional relationship CHAIN with a gap partway through. Both are fixed the same way: add the missing DIRECT relationship between the two entity types that actually need to be connected.

# Key Takeaways

- Promote an attribute to its own entity type when it needs its own attributes, its own relationships, or is shared by many occurrences of the entity currently describing it.
- A relationship needing its own attributes (Viewing, Registers) is a sign it deserves first-class treatment — sometimes as a ternary relationship, since collapsing a three-way fact into two binary relationships can lose the combination information.
- Multiplicity is a (min, max) pair on each end of a relationship — get both numbers right, not just the “many” side.
- Redundant relationships duplicate a fact already derivable via another path — remove the duplicate, don't try to keep two copies in sync.
- Fan traps (two 1:N from one entity) and chasm traps (a path broken by an optional relationship) both look valid on paper but make a real business question unanswerable — both fixed by adding the missing direct relationship.